EngineSquare

보안 관제 및 SOAR 자동 대응 시스템

개인 포트폴리오 정리

개인 저장소: github.com/rykim1217/network-team03

팀 원본: github.com/altdmfk/network-team03

이름: [입력 필요] · 기간: [입력 필요]

1. 프로젝트 개요

웹/API 보안 이벤트를 수집하고 Graylog·Wazuh로 탐지한 뒤 n8n과 API·Discord·pfSense로 자동 대응하는 통합 관제 프로젝트입니다.

구성 흐름

입력	분석	대응
API·공격 트래픽·IoT 이벤트	Middleware WAL → Graylog/Wazuh → 상관분석	n8n → API 차단·키 교체·Discord·pfSense·대시보드

시나리오

시나리오	핵심 기능	개인 검증
1. 브루트포스	로그 버퍼링·401 집계·IP 차단	Alert·n8n·차단 확인
2. API 키 침해	키 폐기·재발급·IP 차단	n8n 직접 실행 성공
3. 킬체인	3단계 상관분석·pfSense 차단	CRITICAL/WARNING 분기 확인

주요 주소

구성	주소/포트
API / Web	5000 / 5001
Middleware	8000 / 9100
Graylog / GELF	9000 / 12201
n8n / MySQL	5678 / 3306
Wazuh / pfSense	1514·1515·443 / 172.16.50.2

2. 설치·실행 절차

새 PC 또는 VM에서 README 순서만 보고 재현할 수 있도록 설치 순서를 고정했습니다.

PowerShell 실행 순서

```
git clone https://github.com/rykim1217/network-team03.git
cd network-team03
Copy-Item .env.example .env
python -m pip install -r requirements.txt -r web-app/requirements.txt
.₩start-enginesquare.ps1
```

실행 전 확인

- Docker Desktop과 MySQL·Middleware·Graylog·Wazuh·n8n이 먼저 실행되어야 합니다.
- 실제 수집 키는 로컬 .env에만 입력하고 .env는 Git에 올리지 않습니다.
- 시나리오 3의 실제 네트워크 차단까지 보려면 pfSense VM과 방화벽 별칭이 필요합니다.
- 실행 후 API /health, 웹 /dashboard, Graylog /alerts, n8n 실행 기록을 순서대로 확인합니다.

설정 파일 관리

- Graylog 집계·알림 규칙, Wazuh 규칙, n8n workflow JSON은 저장소에 포함했습니다.
- Discord는 Incoming Webhook을 사용하며 저장소에는 placeholder만 남겼습니다.
- runtime log, security_events.jsonl, .env는 .gitignore로 제외했습니다.

3. 개인 기여 범위

팀 공통 구현을 개인 단독 구현으로 표현하지 않고, 커밋·수정·검증 기준으로 개인 범위를 분리했습니다.

구분	내용
팀 공통	Flask API, 공통 웹앱, MySQL 모델, Graylog/Wazuh/n8n 기본 구조, Middleware WAL
개인 수정	n8n 시나리오 1~3 Discord Incoming Webhook 전환, workflow 민감정보 placeholder화
개인 반영	시나리오 3 n8n·웹 UI 변경 반영, 루트 테스트를 현재 SecurityStore 구조에 맞게 보완
개인 검증	API/Web smoke, 브루트포스, 2만 건 spike, 시나리오 2 직접 실행, 시나리오 3 CRITICAL/WARNING

커밋 근거

rykim1217 작성 커밋에서 `문구수정`, `n8n파일 교체`, Discord 봇/임베드 알림 최신화, 시나리오 3 workflow·웹 UI 반영 내역을 확인할 수 있습니다. 팀 원본 저장소는 README와 함께 출처로 연결했습니다.

보안 검사

- tracked .env: 없음
- 실제 Discord webhook URL: 없음
- 구형 Discord Bot 토큰·Authorization: 없음
- 로컬 .env·runtime log·security_events.jsonl: ignore 확인

4. 실행·검증 결과

검증	결과	판정
API /health	200, status=ok	완료
Web smoke	200 응답	완료
루트 테스트	14 passed, 2 warnings	완료
Scenario 1	Graylog Alert → n8n 성공 → IP 차단	완료
Scenario 2 직접 실행	키 폐기·재발급·IP 차단·알림 노드 진행	완료
Scenario 3 CRITICAL	pfSense 별칭·인시던트 확인	완료
Scenario 3 WARNING	CRITICAL 격상 없음	완료

2만 건 스파이크 실측

지표	값
최종 received / forwarded	20,005 / 20,005
dropped / DLQ	0 / 0
최종 queue lag	0
peak queue lag	19,051
peak outbound	200 / limit 500

주의: 스크립트의 sent 20,004와 최종 received 20,005가 1건 다릅니다. 테스트 중 백그라운드 이벤트 1건이 합류한 것으로 판단하여 엄격한 exact-count 검사는 실패했지만, 최종 drain·유실 0·lag 0은 확인했습니다.

5. 문제 해결과 개선

Middleware metrics 지연

문제: 첫 2만 건 테스트 전에 /metrics가 멈춤 → 원인: ingestor/worker 정체 → 해결: 두 컨테이너 재시작 후 health·metrics 확인 → 결과: 최종 lag 0, dropped 0.

Discord 방식 불일치

문제: 팀 공용 봇 방식과 개인 채널이 혼용됨 → 원인: n8n 노드 설정 혼합 → 해결: Incoming Webhook으로 통일하고 저장소에는 placeholder만 유지 → 결과: 개인 채널 대상 알림 구조로 정리.

시나리오 2 production webhook

n8n 편집기 직접 실행은 성공했습니다. 다만 Graylog가 호출하는 운영용 POST /webhook/scenario2-api-key-compromised에서 404가 한 번 관찰되어, 제출 전 workflow Active 상태와 Graylog URL을 다시 맞춰야 합니다.

느낀 점

보안 자동화는 탐지 규칙만 만드는 일이 아니라 수집·분석·대응·복구·비밀정보 관리까지 실제 환경에서 끝까지 확인해야 완성됩니다. 다음 개선으로는 run id를 도입해 백그라운드 이벤트가 대량 테스트 수량에 섞이지 않도록 하겠습니다.

6. 제출 전 체크리스트

- 이름과 정확한 프로젝트 기간 입력
- 개인 GitHub 저장소에 README·portfolio·workflow·규칙 파일 push
- API health, 대시보드, Graylog, n8n 성공 화면 캡처
- 시나리오 1 또는 3 재현 화면 캡처
- pfSense 차단 별칭과 복구 화면 캡처
- 2만 건 최종 metrics에서 dropped=0, lag=0 캡처
- GitHub에서 .env·실제 키·토큰·웹훅이 보이지 않는지 최종 검색
- 시나리오 2 production webhook 활성화 후 최종 smoke test
- 이력서·자기소개서 초안의 [입력 필요] 부분 수정

제출 문구

README대로 설치 → 동작 확인 → 담당 작업·개선점 정리 완료

첨부 문서

portfolio/개인_포트폴리오_정리.md · portfolio/검증_기록.md · portfolio/이력서_프로젝트_항목.md · portfolio/자기소개서_기본본.md